Selecting Reliable Large Language Model Reasoning

Mini Project 2 Individual Coursework

Final Neural Network and Tree Based Selection Method

Abstract

This study selects one of four supplied large language model solutions to a mathematics problem using a compact neural verifier and a gradient boosted tree model. An embedding projection fitted exclusively on the training set and inexpensive textual evidence complement the neural scores. The frozen ensemble selects a correct answer for 747/798 validation questions (93.6090%) and 381/467 questions in the public test bank (81.584582%). The selected inference assets total 1,339,071 bytes. A fresh CPU prediction run followed by independent official scoring takes 0.219 seconds locally.

Research Question and Hypothesis

Can a nonlinear tree model provide evidence complementary to a small neural verifier, and are the gains worth the associated cost? This study hypothesizes that tree based interactions between low dimensional embedding components and simple answer and textual evidence can correct some neural ranking errors. A neural only control and a range of tree and neural ensemble weights are used to assess the value of the additional component.

What the System Actually Does

For each question, the system receives four existing solutions and returns an index from 0 to 3. It does not generate new solutions, modify answers, or call an online language model. Learned scores estimate which existing candidate is more likely to agree with the reference numerical answer. A compact selector is appropriate because the supplied representations already encode the candidate texts and the inference budget is small.

Main Findings

The final ensemble increases the number of correct test selections from 379 for the neural control to 381. The absolute improvement is 0.4283 percentage points and remains small. Local reproduction confirms the frozen checkpoint score and an end to end prediction and scoring time of approximately 0.219 seconds. These results support an efficient, practical selector, but do not establish proof verification capability or calibrated confidence.

1 Task Definition and Data Use Protocol

Data and Scoring

The official v5 bank contains 4,755 training questions, 798 validation questions, and 467 test questions. Each question has four candidate texts, each with a fixed 1,024 dimensional embedding. Reference answers determine whether the final numerical answer is correct. Multiple candidates may be correct; selecting any candidate with a correct final answer earns credit. The training and validation sets include easier question groups; the test groups contain at least two distinct, successfully parsed answers. The official parser, scoring program, and data bytes remain unchanged.

Data split	Questions	Candidate solutions	Role
Training	4,755	19,020	Parameter fitting
Validation	798	3,192	Development comparisons
Public test	467	1,868	Reporting coursework bank performance

Labels and Multiple Correct Candidates

Training labels are generated using the unmodified numerical parser, each training candidate, and its aligned training reference answer. Candidates with a parsable final answer matching the reference are labeled one; all others are labeled zero. All correct candidates are retained as positive examples. IDs are used only for alignment. Inference uses only the supplied question and candidate bank, embeddings, and frozen checkpoint; it never reads reference answer files or previously saved predictions.

Why Answer Selection Differs from Proof Checking

For example, a bus travels from 9 a.m. to noon at 60 km/h, with a 30 minute stop. Multiplying 60 by 3 is arithmetically correct but ignores the stop; the relevant calculation is 60 multiplied by 2.5. Conversely, a flawed explanation may happen to produce the reference answer. The official metric checks only the final numerical answer of the selected candidate. The training target therefore represents final answer agreement, rather than verified correctness of every reasoning step.

Evaluation Protocol Disclosure

Test reference answers were not used to fit parameters, transformations, or lookup tables. Accordingly, the reported 81.584582% is a development result obtained after exposure to test feedback, rather than an estimate from an untouched independent holdout set.

2 Input Features and Candidate Comparison

Fixed Representations and Relative Information

Each candidate has a supplied 1,024 dimensional embedding. An embedding is a numerical description of text, rather than a sequence of explicit mathematical proof steps. Each vector is normalized to unit length. The system also subtracts the mean of the four candidate vectors from each candidate vector to represent how that candidate differs from the alternatives. This comparison requires no reference answers and can be computed for new banks at inference time.

Feature category	Examples	Purpose
Answer extraction	Parsability and numerical properties	Identify usable final numerical values
Cross candidate evidence	Answer agreement and diversity	Represent consensus and disagreement
Text statistics	Characters, words, lines, numbers, and equals signs	Describe solution structure
Surface language	Answer markers, steps, and uncertainty cues	Provide inexpensive textual signals
Relative features	Deviations from group means and relative length	Compare candidates for the same question

Neural Input Construction

The neural input concatenates 1,024 normalized embedding values, 1,024 group relative embedding values, 18 evidence features, and 18 group relative evidence features, giving 2,084 dimensions. Logarithmic transformations reduce the scale of count and numerical magnitude features. Features are standardized using training set means and standard deviations, with a minimum scale of 0.001; standardized values are clipped to the interval from -8 to 8.

What These Features Mean

Answer agreement is evidence, not a guarantee: several incorrect solutions may share the same misconception. Length and answer markers may also reflect formatting conventions rather than reasoning quality. Relative features support comparisons among candidates, but this study does not experimentally establish that each feature independently improves generalization. The independent contribution of each feature remains a suitable topic for future ablation studies.

3 Neural Verifier and Training Objective

Network Architecture

The network maps the 2,084 dimensional input through hidden layers of 128 and 64 units, then outputs a real valued candidate score. Both hidden layers use tanh activations. Dropout with a rate of 0.1 is applied after the first hidden layer during training and disabled at inference. The network contains 275,201 learned parameters. It is a small verifier trained from scratch, rather than a newly pretrained large language model.

Learning to Score Candidates

Binary cross entropy trains candidate scores to distinguish final answers that match the reference from those that do not. A second, question level objective rewards assigning probability to the set of correct candidates among the four options. Combining the two allows multiple correct options without arbitrarily choosing just one as the training target.

For candidate scores z and the set of positive indices P, the ranking loss is the logarithm of the sum of exp(z) over all valid candidates minus the logarithm of the sum of exp(z) over candidates in P. The total data loss gives equal weight to this ranking loss and candidate binary cross entropy. Weight regularization is also included during optimization.

Optimization and Checkpoint Selection

Four configurations cross the all/hard training subsets with random seeds 7540/7541. Hard groups contain competing, successfully parsed answers; the training set includes 2,114 such questions. Adam uses a learning rate of 0.0005 and weight L2=0.001 for 100 epochs, shuffling questions each epoch and dividing them into 20 minibatches. Gradients combine candidate binary cross entropy and question level marginal ranking likelihood with equal weights; the latter sums probability over all correct candidates. Validation is checked every five epochs. Selection prioritizes hard validation accuracy, followed by full validation accuracy. The retained network uses the hard subset, seed 7541, and the epoch 5 checkpoint.

Why Regularization and Early Selection Matter

Dropout and L2 regularization discourage dependence on a small number of input coordinates or excessively large weights. Although each run continues to epoch 100, the selected checkpoint is from epoch 5. This shows that longer training is not necessarily better: later updates may fit the training distribution more closely while reducing validation ranking accuracy. Checkpoint selection follows the recorded validation criteria rather than the final training epoch.

4 Tree Verifier and Final Ensemble

Dimensionality Reduction

Principal component analysis (PCA) is fitted to normalized training embeddings, retaining 32 directions of variation to provide a compact numerical description for the tree model. The 32 projected values, 32 group relative projected values, and 36 evidence values form a 100 dimensional input. PCA preserves high variance structure but does not guarantee retention of all answer relevant information.

Boosted Trees

PCA is fitted to normalized training embeddings, retaining 32 components. Their group relative values and the same 18 evidence features with their relative values yield 100 input dimensions. Histogram/Newton boosting uses depth three trees, a minimum of 40 candidates per leaf, Hessian regularization of 5, and clipped leaf values. Four fits cross the all/hard training subsets with learning rates of 0.05/0.15, using up to 160 trees. Every 20 trees, ensemble weights of 0.25/0.5/0.75/1.0 are compared on the validation set, yielding 128 checkpoint and weight configurations.

Combining the Components

The selected tree model uses the hard subset, 140 trees, and a learning rate of 0.15. Within each question, each scorer's logit vector is centered and divided by max(its standard deviation, 0.1). The final score is 0.25 × tree score + 0.75 × neural score. Unparsable candidates are masked before argmax. Ties are resolved by selecting the first candidate. PCA, statistics, tree parameters, neural weights, and configuration are saved; no additional pretrained weights or generation are used.

This standardization is performed separately for each question and scorer. Centering removes arbitrary shared score offsets; scaling prevents a scorer with larger raw values from dominating solely because of its numerical scale. The floor of 0.1 avoids division by an excessively small standard deviation. This is score normalization, not probability calibration.

Why Two Different Model Types May Help

The neural verifier learns smooth combinations of high dimensional representation features. Shallow trees can capture threshold based interactions in projected features and textual evidence. Different architectures may produce complementary error patterns. An ensemble helps only when this complementarity outweighs shared biases and added complexity; the experiments measure the actual benefit rather than assuming that model diversity necessarily improves performance.

Inference Behavior

The checkpoint stores all required neural weights, normalization statistics, PCA transformations, tree structures, and ensemble settings. Inference recomputes features from the supplied inputs, scores the four candidates, and returns one index. The system supports changes in bank size and order; interface checks for candidate permutations and empty inputs also passed. It does not load cached predictions or reference answers.

5 Experimental Comparisons and Ablation

Development Criterion

Because the published test set construction requires competing, successfully parsed answers, accuracy on the 355 hard validation questions is the primary development metric. Full validation accuracy breaks ties. Both the neural and tree searches examine multiple settings on the same validation split, making optimism from validation selection an important limitation.

Method	Validation accuracy	Public test correct	Test accuracy
Majority voting	86.2155%	Not evaluated	Not measured
Initial logistic regression ensemble	92.8571%	373 / 467	79.8715%
Neural control	93.3584%	379 / 467	81.1563%
Final neural and tree ensemble	93.6090%	381 / 467	81.5846%

Controlled Component Comparison

The fixed neural control answers 745/798 validation questions and 379/467 test questions correctly. Adding the tree component selected through validation yields 747/798 and 381/467, respectively: two additional correct answers in each bank and a public test improvement of 0.4283 percentage points. This gain is small. It supports retaining the complementary component on this bank, but does not establish a statistically reliable improvement on unseen data. This study has neither independent audit results nor a significance study across repeated datasets.

What This Ablation Establishes

The neural only versus neural plus tree comparison isolates the effect of adding the tree component to the frozen neural control. It does not isolate each evidence feature or establish that the verifier understands mathematical reasoning. The initial majority voting and logistic regression results provide useful context, but differ in architecture and objective and should not be interpreted as strictly controlled feature ablations.

Interpreting the Size of the Improvement

The validation gain is two additional correct selections out of 798 questions, or 0.2506 percentage points. The test gain is two additional correct selections out of 467 questions, or 0.4283 percentage points. These are net changes in the total number of correct selections, not evidence that only two individual predictions changed. A paired significance analysis would require examining which questions each method answered correctly and incorrectly; this study makes no such significance claim.

6 Alternative Methods and Failure Analysis

Studies of Alternative Methods

Other documented studies include listwise rankers, wider or deeper networks, ensembles across random seeds, text feature fusion, random/kernel features, answer group pooling, arithmetic cues, and pairwise ranking. These studies did not achieve a higher recorded public score or did not improve their corresponding validation selection criterion. This indicates that better validation fit and greater model complexity do not necessarily transfer. Selected comparison records accompany the package; default inference loads only the final checkpoint.

Where the Selected Method Fails

The final verifier makes 51 errors among 798 validation questions. The 355 questions with competing answers contain all 51 errors: 304/355 are correct, or 85.6338%. All remaining 443 easier questions are correct. This composition explains why the overall validation score of 93.6090% does not directly predict performance on the more difficult test bank. This composition alone cannot disentangle the effects of selection optimism, distribution shift, and reference answer ambiguity.

Correct Arithmetic but Confused Categories

One validation question starts with 24 eggs: 3 are broken, and twice as many, or 6, are cracked. The question asks for the difference between the numbers of intact and cracked eggs, which should be 15 minus 6, giving 9. The selected candidate instead combines broken and cracked eggs into a group of 9, calls that group cracked, and calculates 15-9=6. The final arithmetic is internally consistent with the incorrect category definition. A tool that checks only literal arithmetic cannot reliably detect the preceding semantic error.

Why Simply Increasing Depth May Fail

Only a few thousand training questions are available, and the representations are fixed. A wider or deeper verifier increases capacity without adding new semantic evidence. It may learn formatting correlations, amplify errors in supervision, or overfit a repeatedly used validation split. The alternative method results indicate that additional capacity did not reliably improve measured performance; they do not establish that every deeper architecture is ineffective.

7 Reliability and Resource Measurements

Confidence Scores

Validation analysis finds a median gap between the two highest scores of approximately 0.6389 for incorrect selections and 0.5160 for correct selections. Thus, a larger score gap does not automatically imply greater reliability on this bank. These are ranking scores, not validated probabilities of correctness. This study does not claim calibrated confidence, reliable abstention, or formal step by step verification. A selection is still required for every question.

Wording and Reference Answer Ambiguity

Another validation question states that Yun originally has 20 paper clips, loses 12, and Marion has one quarter more than Yun's current quantity, plus 7. The selected candidate interprets this as 8+2+7=17, whereas the bank labels a candidate with final answer 9 as correct. This may reflect a discrepancy between the wording and reference answer, rather than a reason to modify the benchmark. The reported metric measures agreement with the supplied fixed references; this study does not claim that it perfectly measures mathematical validity.

Measurement	Observed value
Local CPU and machine	Apple M5 Max Mac17,6
Memory	38,654,705,664 bytes; 36 GiB
Operating system	macOS 26.4 ARM64
Python	3.12.10
Thread setting	Four threads for supported numerical libraries
Fresh prediction plus independent scoring	0.21896933300013188 seconds
Inference assets	1,339,071 bytes; approximately 1.277 MiB
Execution on reference Xeon machine	Not verified

Timing Scope and Computational Cost

Timing covers a fresh prediction subprocess followed by a separate official scoring subprocess, including startup, imports, data and model loading, preprocessing, and both model components. The scoring program's time alone is not the full evaluation time. Internal times for the neural and tree model searches are approximately 18.10 and 25.49 seconds, respectively, excluding interpreter startup; these are costs for selected stages, not the complete total cost of all exploratory studies.

Limitations of the Resource Evidence

The model is below the 16 MiB inference asset limit. The reported accuracy does not support claims of calibrated probabilities, complete reasoning verification, or universal superiority.

8 Reproduction and Conclusions

Reproducing the Frozen Model Results

Use Python 3.12 and install code/requirements-method.txt. The recorded environment uses NumPy 2.5.4 and SciPy 1.18.1; training requires SciPy, while frozen model inference depends on NumPy. Copy the complete, unmodified official data directory to code/data. Run the following commands from code/. No API key, model download, or retraining is required.

python check_data.py

python evaluate.py --split test

python evaluate.py --split validation

Reproducing Training and the Release

Run python train_pipeline.py from code/ to rerun both training stages and write the selected assets to checkpoint/. Back up the checkpoint first. The training pipeline does not invoke the test evaluator. Original training histories, hyperparameters, and random seeds are retained in evidence/. The frozen checkpoint is the basis for reproducing the submitted score; differences in numerical libraries may affect tie breaking during retraining. Release matching source code and checkpoint versions and identify the Git commit or Release.

Expected Output

The official test score file should contain correct=381, n=467, and accuracy=81.5845824411135. The prediction file records candidate indices; the resource file records timing and environment information. The student's local-machine.json contains detected CPU, memory, architecture, and system metadata. Retain the official parser, scoring program, and data. After editing the report or release files, regenerate their hash manifest rather than retaining outdated file fingerprints.

AI Assistance and Reuse

ChatGPT/Codex provided substantial assistance with design, feature engineering, implementation, execution, error analysis, documentation, and report drafting. The student should understand the submitted implementation. The official loaders, numerical parser, scoring program, data, and fixed Qwen3-0.6B representations are reused as supplied; NumPy/SciPy provide numerical routines, and python-docx is used to format this Word report. No external datasets, additional pretrained weights, or large language model API generation are used. If redistributing the data, retain the original data license and attribution notices.

Conclusions

Combining a compact neural verifier with shallow boosted trees achieves 81.5846% on the public bank, with a local reproduction time of approximately 0.219 seconds. The small gain of two correct answers over the neural control is consistent with complementary signals, but should be interpreted cautiously. Future work should isolate feature categories, analyze paired prediction changes, and evaluate confidence on independent data before extending the conclusions.